

Laboratorio di Programmazione

Lezione 9: Array e Slice (2)

1 Qual è l'output?	3
2 Qual è l'output?	4
3 Qual è l'output?	5
4 Qual è l'output?	6
5 Qual è l'output?	7
6 Qual è l'output?	8
7 Append e copy	9
8 Filtra e moltiplica	10
9 Minimo, massimo e valor medio (1)	11
10 Minimo, massimo e valor medio (2)	12
11 Operazioni	13
12 Date (1)	14
13 Fattoriale	15
14 Prodotto interno	16
15 Somma di prodotti pari	17
16 Filtra voti	18
17 Numeri casuali	19
18 Filtra testo	20
19 Date (2)	21
20 Somma interi unici	22
21 Controlla sequenza	23

22 Primi	24
23 Matrici.....	25
24 Trasposizione di matrice.....	26
25 Gestione di matrici	27
26 Prodotto di matrici	28

1 Qual è l'output?

```
1 package main
2 import "fmt"
3
4 func main() {
5     a := [...]string{5:"E", 3:"C"}
6
7     fmt.Printf("Tipo: %T - Valore: %v\n", a, a)
8
9     for i := range a {
10         fmt.Println("Indice", len(a)-1-i, " - Valore:", a[len(a)-1-i])
11     }
12     fmt.Println()
13
14     for i:=len(a)-1; i>=0; i-- {
15         fmt.Println("Indice", i, " - Valore:", a[i])
16     }
17     fmt.Println()
18
19     for i:=0; i<len(a); i++ {
20         fmt.Println("Indice", i, " - Valore:", a[i])
21     }
22     fmt.Println()
23 }
```

2 Qual è l'output?

```
1 package main
2 import "fmt"
3
4 func main() {
5     var a [6]int
6
7     for i := range a {
8         a[i] = i
9     }
10
11     var b []int
12     b = a[:]
13
14     for i := range b {
15         b[i] = i * 2
16     }
17
18     fmt.Println(a)
19 }
```

3 Qual è l'output?

```
1 package main
2 import "fmt"
3
4 func main() {
5     var a = [6]int{1, 2, 3, 4, 5, 6}
6     stampa(a)
7     a = modifica(a)
8     stampa(a)
9     modificaConPuntatore(&a)
10    stampa(a)
11 }
12 func stampa(a [6]int) {
13     for _, v := range a {
14         fmt.Print(v, " ")
15     }
16     fmt.Println()
17 }
18 func modifica(a [6]int) [6]int{
19     for i := range a {
20         a[i] *= 2
21     }
22     return a
23 }
24
25 func modificaConPuntatore(a* [6]int) {
26     for i := range (*a) {
27         (*a)[i] *= 2
28     }
29 }
```

4 Qual è l'output?

```
1 package main
2 import "fmt"
3
4 func main() {
5     var a = []int{1, 2, 3, 4, 5, 6}
6     stampa(a[:])
7
8     b := a[:]
9     b = modifica(b)
10    stampa(b)
11 }
12
13 func stampa(sl []int) {
14     for _, v := range sl {
15         fmt.Print(v, " ")
16     }
17     fmt.Println()
18 }
19
20 func modifica(slIn []int) (slOut []int) {
21     slOut = slIn
22     for i := range slOut {
23         slOut[i] *= 2
24     }
25     return
26 }
```

5 Qual è l'output?

```
1 package main
2 import "fmt"
3
4 func main() {
5     b := []int{1,2,3,4,5}
6     stampa(b)
7     eliminaUltimoElemento(b)
8     stampa(b)
9 }
10
11 func stampa(sl []int) {
12     for _, v := range sl {
13         fmt.Print(v, " ")
14     }
15     fmt.Println()
16 }
17
18 func eliminaUltimoElemento(sl []int) {
19     sl = sl[:len(sl)-1]
20 }
```

6 Qual è l'output?

Inserendo **da riga di comando** i valori

1 a 5.6 ciao true

cosa stampa il programma?

```
1 package main
2 import (
3     "os"
4     "fmt"
5 )
6
7 func main() {
8     fmt.Printf("TIPO os.Args: %T\n", os.Args)
9     for i, v := range os.Args {
10         fmt.Printf("os.Args[%d]: TIPO = %T - VALORE = %s\n", i, v, v)
11     }
12 }
```

7 Append e copy

Implementate le funzioni

```
AppendInt(S []int, T []int) []int
```

```
CopyInt(S []int, T []int) int
```

in modo che si comportino come le funzioni `append()` e `copy()` di Golang. Inoltre adattatele a slice di altri tipi (rune, stringhe, ...).

8 Filtra e moltiplica

Scrivere un programma che legga da **riga di comando** una sequenza di parametri, e stampi a video il prodotto dei parametri che rappresentano numeri interi.

Esempio d'esecuzione:

```
$ go run prodotto.go 4 3  
12
```

```
$ go run prodotto.go ciao -1 ciao 6  
-6
```

```
$ go run prodotto.go 1 3 a 5 ciao 2  
30
```

9 Minimo, massimo e valor medio (1)

Scrivere un programma che legga da **riga di comando** una sequenza di valori interi e stampi a video il loro minimo, il loro massimo e la loro media, calcolati usando opportune funzioni:

- Minimo(sl []int) int
- Massimo(sl []int) int
- Media(sl []int) float64

Esempio d'esecuzione:

```
$ go run min_max_media.go 1 2 3 4
```

```
Minimo: 1
```

```
Massimo: 4
```

```
Valore medio: 2.50
```

```
$ go run min_max_media.go -1 10 6
```

```
Minimo: -1
```

```
Massimo: 10
```

```
Valore medio: 5.00
```

```
$ go run min_max_media.go -1 -2 -3
```

```
Minimo: -3
```

```
Massimo: -1
```

```
Valore medio: -2.00
```

10 Minimo, massimo e valor medio (2)

Adattate la soluzione dell'esercizio precedente aggiungendo una funzione `LeggiNumeri() []int` che restituisce i valori interi che appaiono sulla riga di comando.

Esempio d'esecuzione:

```
$ go run min_max_media.go 1 ciao 2 pippo 3 4
Minimo: 1
Massimo: 4
Valore medio: 2.50
```

```
$ go run min_max_media.go -1 10 6 fine
Minimo: -1
Massimo: 10
Valore medio: 5.00
```

```
$ go run min_max_media.go tre -1 numeri: -2 -4
Minimo: -3
Massimo: -1
Valore medio: -2.33
```

11 Operazioni

Usate `append` e lo *slicing* per implementare le seguenti operazioni (`a` e `b` sono due slice generiche):

1. Accodare `b` ad `a`
2. Copiare `b` su `a`
3. Cancellare l'elemento `a[i]` da `a`
4. Cancellare gli elementi `a[i], \dots, a[j-1]` da `a`
5. Estendere `a` di k elementi
6. Inserire `b` fra `a[i]` e `a[i+1]`

12 Date (1)

Scrivere un programma che:

- legga da **standard input** un testo su più righe;
- termini la lettura quando viene inserita da standard input una riga vuota ("").

Ogni riga è una stringa senza spazi che codifica una data in uno dei seguenti formati:

1. aaaa/m/g
2. aaaa/m/gg
3. aaaa/mm/g
4. aaaa/mm/gg

Il programma deve poi stampare le date lette, nel formato aaaa-mm-gg. Il programma deve definire ed utilizzare le seguenti funzioni:

- `LeggiTesto() []string` che restituisce il testo letto (inclusi i newline)
- `Formatta(s string) string` che converte una data, scritta in uno dei formati sopra, nel formato aaaa-mm-gg.

Esempio d'esecuzione:

```
$ cat test
2019/04/03
2019/6/4
2019/07/5
2019/4/05
$ go run date.go < test
2019-04-03
2019-06-04
2019-07-05
2019-04-05
```

13 Fattoriale

Il fattoriale di un intero $k \geq 1$, indicato con $k!$, è definito come:

$$k! = 1 \cdot 2 \cdot \dots \cdot (k - 1) \cdot k$$

Scrivere un programma che legge da riga di comando un intero n e stampi i fattoriali di tutti gli interi nell'insieme $\{1, \dots, n\}$. Il programma deve usare una funzione `Fattoriale(k int) int` che calcola il fattoriale $k!$ di k .

Esempio d'esecuzione:

```
$ go run fattoriale.go 2
```

```
Fattoriali: [1 2]
```

```
$ go run fattoriale.go 3
```

```
Fattoriali: [1 2 6]
```

```
$ go run fattoriale.go 10
```

```
Fattoriali: [1 2 6 24 120 720 5040 40320 362880 3628800]
```

14 Prodotto interno

Scrivere un programma che legga da standard input due righe, ognuna delle quali contiene un numero arbitrario di valori in virgola mobile. Se le due righe contengono un numero diverso di valori, il programma esce con un errore. Altrimenti, siano $\mathbf{u} = (u_1, \dots, u_n)$ e $\mathbf{v} = (v_1, \dots, v_n)$ i vettori definiti dai valori immessi. Il programma deve calcolare e stampare il prodotto interno di $\mathbf{u}$ e $\mathbf{v}$, cioè:

$$\mathbf{u} \cdot \mathbf{v} = \sum_{i=1}^n u_i \cdot v_i$$

A tal scopo implementate le seguenti funzioni:

- `LeggiVettore() []float64` che legge un vettore da una riga dello standard input.
- `Prodotto(u, v []float64) float64` che calcola il prodotto interno $\mathbf{u} \cdot \mathbf{v}$ di $\mathbf{u}$ e $\mathbf{v}$.

Esempio d'esecuzione:

```
$ go run inner_product.go
0.5 0.2
0.3 0.5
prodotto interno: 0.25
```

```
$ go run inner_product.go
0.6 0.2 -0.2
-0.3 1.0 0.1
prodotto interno: 0
```


15 Somma di prodotti pari

Scrivere un programma che legga da riga di comando una insieme di numeri interi, e stampa la somma dei prodotti *pai* ottenibili moltiplicando una coppia di interi in input. A tale scopo usate una funzione `Prodotti(S []int) []int` che restituisce i prodotti delle coppie di `S`.

Esempio. Se l'insieme di numeri in input è

4, 3, 5, 8

allora l'insieme dei prodotti ottenibili è

12, 20, 32, 15, 24, 40

e il sottoinsieme dei prodotti pari è

12, 20, 32, 24, 40

Il programma dovrà quindi stampare 128 (che è la loro somma).

Esempio d'esecuzione:

```
$ go run prodotti_pari.go 1 2 3 4 5 6
```

La somma è 152

```
$ go run prodotti_pari.go 1 2 3 4 5
```

La somma è 62

16 Filtra voti

Scrivere un programma che:

- legga da riga di comando una sequenza di valori (i valori numerici interi che compaiono all'interno della sequenza rappresentano voti in una scala di valutazione tra 0 e 100; i valori numerici interi superiori a 60 corrispondono a voti sufficienti);
- stampi a video le due sottosequenze di valori numerici interi che corrispondono rispettivamente a voti insufficienti e sufficienti.

A tal scopo scrivete le seguenti funzioni, di ovvio comportamento:

- LeggiNumeri() (numeri []int)
- FiltraVoti(voti []int) (sufficienti, insufficienti []int)

Esempio d'esecuzione:

```
$ go run filtro.go 80 75 60 55  
Voti sufficienti: [80 75 60]  
Voti insufficienti: [55]
```

```
$ go run filtro.go 100 98 59 40  
Voti sufficienti: [100 98]  
Voti insufficienti: [59 40]
```

17 Numeri casuali

Scrivere un programma che legge da riga di comando un numero intero $n \leq 100$, e poi generi una sequenza di numeri casuali interi in $[0, 100]$, interrompendo la sequenza appena dopo aver generato un numero inferiore a n . A questo scopo implementate una opportuna funzione `Genera(soglia int) []int`. Ricordate che potete generare numeri casuali così:

```
1 rand.Seed(int64(time.Now().Nanosecond())) // una volta sola
2 numeroGenerato := rand.Intn(100) // ogni volta che si vuole un intero casuale
   fra 0 e 100
```

Esempio d'esecuzione:

```
$ go run numeri_random.go 20
21 72 44 64 30 13
```

18 Filtra testo

Scrivere un programma che legge da standard input un testo arbitrario, anche su più righe, finché non incontra EOF, e ristampi poi le righe in cui compaiono cifre. A tal scopo implementate le seguenti funzioni:

- `LeggiTesto()` `[]string` che ritorna le righe lette.
- `ContieneCifre(s string)` `bool` che decide se `s` contiene cifre; vedete `unicode.IsDigit`.
- `FiltraTesto(testo []string)` `[]string` che restituisce il sottoinsieme di righe di testo che contengono cifre.

```
$ go run filtra_testo.go
riga senza cifre
riga con 1 cifra
2a riga con 2 cifre
nessuna cifra
>>> Testo filtrato:
riga con 1 cifra
2a riga con 2 cifre
```

19 Date (2)

Adattate la soluzione di “Date” in modo da stampare le date in ordine cronologico. Per fare questo potete usare `sort.Strings(a []string)`.

Esempio d’esecuzione:

```
$ cat test
2019/04/03
2019/6/4
2019/07/5
2019/4/05
5/5/2019
05/4/2019
7/05/2019
07/09/2019
07/09/2018

$ go run date.go < test
2018-09-07
2019-04-03
2019-04-05
2019-04-05
2019-05-05
2019-05-07
2019-06-04
2019-07-05
2019-09-07
```

20 Somma interi unici

Scrivere un programma che legga da riga di comando una sequenza di argomenti, e stampi a video la somma di quelli che rappresentano numeri interi e che compaiono una sola volta. A tal scopo implementate le seguenti funzioni:

- `LeggiNumeri() []int` che restituisce gli interi presenti sulla riga di comando.
- `Occorrenze(seq []int, n int) int` che conta quante volte `n` appare in `seq`.

Esempio d'esecuzione:

```
$ go run somma_unici.go 1 2 % 4 3 2 1 5  
12
```

```
$ go run somma_unici.go 4 3 5 4 2 2 3 2  
5
```

```
$ go run somma_unici.go 1 2 sarà zero 1 2  
0
```

```
$ go run somma_unici.go 1 2 3 2 2 2  
4
```

```
$ go run somma_unici.go che 10 4 7 12 4 12 sfortuna  
17
```

21 Controlla sequenza

Scrivere un programma che legge da standard input una stringa arbitraria e verifichi se soddisfa le seguenti condizioni:

- ogni coppia di cifre è separata da almeno un carattere che non è una cifra.
- la sequenza formata dalle cifre è ordinata per valori non crescenti.

Ad esempio, nella stringa &4&\$4%mamma5!6mia6cosa1succede0 appaiono le cifre 4 4 5 6 6 1 0 e pertanto la seconda condizione è violata (la prima invece è soddisfatta).

Esempio d'esecuzione:

```
$ go run controlla_sequenza.go  
ab c8dè f6asa 2sd0  
Tutto OK.
```

```
$ go run controlla_sequenza.go  
Èbc87d8e5a#4$d1e  
Cifre non separate!
```

```
$ go run controlla_sequenza.go  
a° c9 d8e 6 àà7 sd1e  
Cifre non ordinate!
```

22 Primi

Scrivere un programma che legga da riga di comando un intero $n \geq 2$ e stampi, in ordine crescente, tutti i numeri primi ottenibili rimuovendo da n al più 3 cifre consecutive. Ad esempio, con $n = 5899$, i numeri ottenibili rimuovendo al più 3 cifre consecutive sono 5, 9, 58, 59, 99, 589, 599, 899, 5899. Fra questi, i primi sono 5, 9, 59, 599. Nella soluzione implementate una funzione `Primo(n int) bool` che decide se n è primo. Per ordinare una slice di interi usate la funzione `sort.Ints(a []int)`.

Esempio d'esecuzione:

```
$ go run primi.go 5899
```

```
5
59
599
```

```
$ go run primi.go 5894457
```

```
4457
5857
5897
594457
```

```
$ go run primi.go 10113
```

```
13
13
101
103
113
113
113
1013
1013
```

```
$ go run primi.go 2468
```

```
2
```


23 Matrici

Scrivete un programma che legga da standard input una matrice $n \times m$ di valori in virgola mobile, riga per riga, terminando quando incontra una riga vuota (notare quindi che n, m non sono noti in anticipo, ma vanno calcolati a partire dall'input). Stampate poi la matrice a video. A tale scopo implementate le seguenti funzioni:

- `LeggiMatrice()` `[][]float64` che legge la matrice da standard input.
- `StampaMatrice(A [][]float64)` stampa A, con precisione di 3 cifre dopo la virgola.

Modificate la soluzione in modo da leggere *da riga di comando* la precisione p di stampa.

24 Trasposizione di matrice

Adattate la soluzione dell'esercizio precedente per stampare la *trasposta* della matrice letta. Ricordiamo che la trasposta di una matrice A è la matrice A^T ottenuta “scambiando” righe e colonne; cioè, per ogni $i = 1, \dots, n$ ed ogni $j = 1, \dots, m$,

$$A^T[i, j] = A[j, i]$$

A tale scopo implementate una funzione `Trasponi(A [][]float64) [][]float64`.

Esempio d'esecuzione:

```
% go run trasponi.go
```

```
1 1 1 1 1
```

```
0 0 0 0 0
```

```
1 0
```

```
1 0
```

```
1 0
```

```
1 0
```

```
1 0
```

```
% go run trasponi.go
```

```
0.0 1.0
```

```
-1.0 2.0
```

```
0.0 -1.0
```

```
1.0 2.0
```

25 Gestione di matrici

Scrivete le seguenti funzioni per la gestione di matrici reali $A \in \mathbb{R}^{n \times m}$.

- NuovaMatrice(n, m int) `[][]float64` che restituisce una slice bidimensionale che rappresenta una matrice. Le slice *devono essere allocate*, in modo che gli elementi siano già accessibili (e ovviamente inizializzati al loro zero-value).
- Dim(A `[][]float64`) (n, m int) che restituisce il numero di righe e colonne di A .
- NNZ(A `[][]float64`) int che restituisce il numero di elementi non nulli (non 0) in A .
- NuovaIdentità(n int) `[][]float64` che restituisce una matrice identità di lato n .
-

Testatele usando il codice degli esercizi precedenti.

26 Prodotto di matrici

Adattate le soluzioni degli esercizi precedenti in modo da leggere *due* matrici, $A \in \mathbb{R}^{n \times m}$ e $B \in \mathbb{R}^{m \times q}$. Se il numero di colonne di A non è uguale al numero di righe di B , il programma deve terminare con un messaggio. Il programma deve poi stampare AB , cioè il *prodotto* di A e B . Ricordate che questa è la matrice definita come segue:

$$(AB)[i, j] = \prod_{h=1, \dots, m} A[i, h]B[h, j]$$

Esempio d'esecuzione:

```
% go run prodotto.go
```

```
Inserisci A:
```

```
0 2 3
```

```
1 4 0
```

```
Inserisci B:
```

```
2 0 0
```

```
0 2 0
```

```
0 0 2
```

```
Il prodotto è:
```

```
0 4 6
```

```
2 8 0
```

```
% go run prodotto.go
```

```
Inserisci A:
```

```
0 2 3
```

```
1 4 0
```

```
Inserisci B:
```

```
2 0 0
```

```
0 2 0
```

```
Dimensioni non compatibili!
```